Spam Message Classification

KNN vs SVM vs Naive Bayes: theory, experiments and recommendation

1. Theory and Definitions (Part A)

1. Conditional probability. P(A|B) = P(A and B) / P(B), the probability of event A given that B has occurred (P(B) > 0). Example: P(spam | message contains a URL).

2. Bayes' theorem. P(y|x) = P(x|y) P(y) / P(x). It converts the likelihood P(x|y) and the prior P(y), which are easy to estimate from labelled data, into the posterior P(y|x) that a classifier needs. Choosing the class with the largest posterior is the Bayes decision rule, which is optimal when the probabilities are right. It also gives confidence scores and allows class priors and costs to be included explicitly.

3. Naive Bayes assumptions. (i) Features are conditionally independent given the class, so P(x|y) is the product of P(x_j|y). (ii) Each feature follows an assumed distribution per class (Gaussian for GaussianNB). (iii) Training and future data come from the same distribution, and the class prior is estimated from the training set.

4a. KNN. A lazy, non-parametric method: it stores the training data and classifies a new point by a majority vote of its K nearest training points under a distance metric (Euclidean, Manhattan, ...). Small K means low bias and high variance; large K the opposite. Features must be scaled.

4b. SVM. Finds the separating hyperplane with the maximum margin; only the support vectors (points on or inside the margin) define it. The parameter C controls how many margin violations are tolerated; kernels (RBF, polynomial) implicitly map the data to a higher-dimensional space to obtain non-linear boundaries.

5. Comparison of the three families.

	Distance-based (KNN)	Probabilistic (Naive Bayes)	Margin-based (SVM)
Idea	Similar points share labels	Model P(y|x) with Bayes' rule	Maximise the margin between classes
Training	None (stores data)	Very fast (means, variances, priors)	Quadratic optimisation, slower
Prediction	Slow (neighbour search)	Very fast	Fast (support vectors only)
Needs scaling	Yes	No	Yes
Output	Vote share	Class probabilities	Signed distance to boundary
Weakness	Irrelevant features, noise, cost at scale	Independence assumption	Tuning, no native probabilities

2. Data and Method (Part B)

The dataset holds 5,200 messages with 12 numeric input features (message length, word count, URL / digit / special-character counts, spam and legit keyword scores, sender activity score, account age, messages sent in 24 h, hour of day, day of week) and the target **spam_label** (973 spam = 18.7%, 4,227 legitimate). *message_id*, *message_text* and *timestamp* were not used as inputs (identifier, raw text summarised by the numeric features, and information already in hour and weekday).

Preprocessing: about 2-3% of three sender columns were missing and were imputed with the training median; features were standardised for KNN and SVM; Naive Bayes used the imputed raw values. The data were split 80/20 with stratification (4,160 train / 1,040 test). Imputer and scaler were fitted on training data only, inside scikit-learn pipelines, to avoid leakage. Hyper-parameters were chosen by 5-fold stratified cross-validation on the training set; the test set was used once.

Important data caveat. Every spam message has at least one special character and no legitimate message has any, so the single rule "spam if num_special_chars > 0" classifies all 5,200 messages correctly. The data are therefore cleanly separable (probably synthetic), and near-perfect model scores should not be read as evidence that real-world spam detection is this easy.

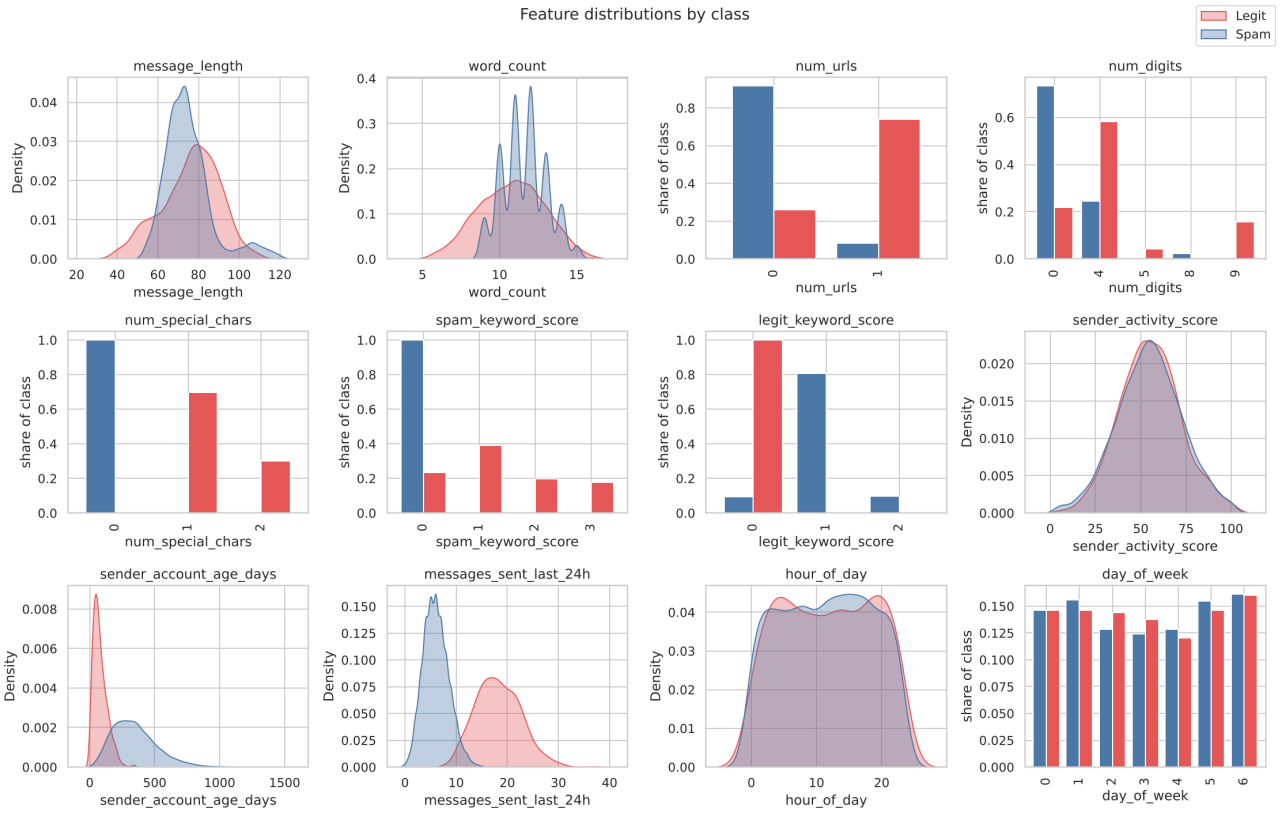


Figure 1. Feature distributions by class. URLs, digits, special characters, keywords, message volume and account age differ strongly; length, activity score, hour and weekday barely differ.

3. Experiments (Parts C-E)

K-Nearest Neighbors

Cross-validated F1 is 1.000 at K = 1 and stays above 0.996 up to K = 41. Euclidean, Manhattan, Minkowski (p=3) and cosine distances all reach F1 = 1.000; Chebyshev is slightly worse (0.9994). Scaling is the dominant factor: without it Euclidean KNN falls to CV F1 0.898 and Manhattan to 0.938, because account age (std about 189 days) swamps small-range features. The tuned KNN makes no test errors, so errors were studied under weaker settings: without scaling, KNN makes 33 errors (24 missed spam, 9 false alarms). Missed spam come from older accounts (about 183 days, vs 79 for spam on average), and false alarms are legitimate messages from very new accounts (about 69 days), because distance is dominated by account age. With K = 41 or Chebyshev, the only errors are 1-2 borderline missed spam.

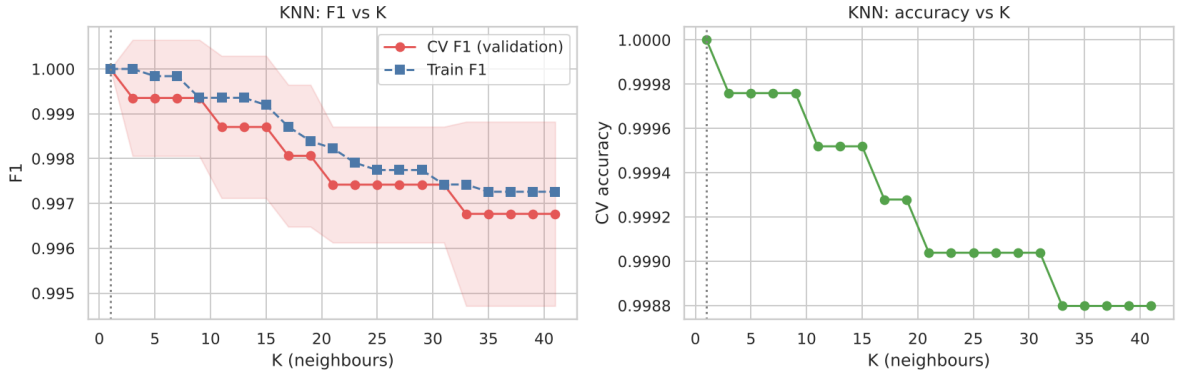


Figure 2. KNN F1 and accuracy against K (5-fold CV on the training set).

Support Vector Machine

Linear, RBF and polynomial kernels all reach CV F1 = 1.0 and perfect test scores; grid search picked linear with C = 0.01 (82 support vectors; RBF 301, polynomial 79). The margin analysis shows the soft-margin

trade-off: at C = 0.001 the margin is wide (width 3.39) with 10.2% of the training data as support vectors; from C = 1 the margin stays at 1.89 with only 10 support vectors and zero training errors, so the data are linearly separable. Against KNN, the SVM has identical accuracy but needs only its support vectors at prediction time, whereas KNN stores and searches all 4,160 training messages.

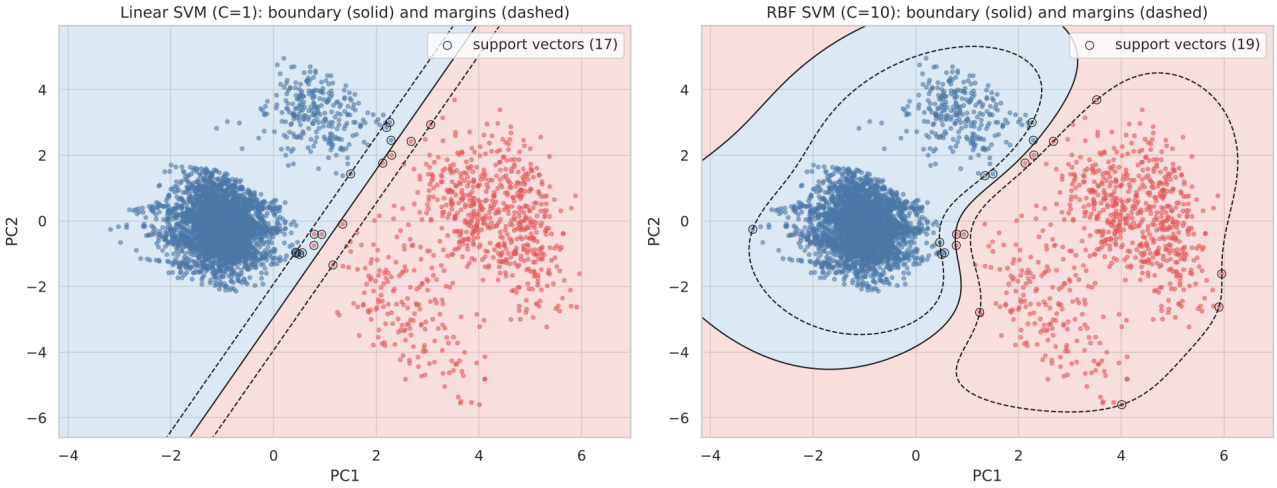


Figure 3. Linear and RBF SVM boundaries (solid) and margins (dashed) on the first two principal components (48.8% of variance; display only). Circled points are support vectors.

Naive Bayes and Bayes' theorem by hand

Gaussian Naive Bayes reaches accuracy 0.999 with precision 0.995 and recall 1.000 (one false alarm). Using training counts, Bayes' rule gives P(spam | URL) = 0.681 (observed 0.682), P(spam | spam keyword) = 0.998 and P(spam | more than 12 messages / 24 h) = 0.957; the naive combination of all three events gives 1.000, matching the observed 1.000 over 396 messages. A hand-written implementation of the Gaussian posterior (log prior + sum of log normal densities, normalised) matches scikit-learn on every one of the 1,040 test messages (largest difference 5e-17). The one error is a legitimate message with a URL and 8 digits whose posterior is P(spam) = 0.557.

4. Model Comparison (Part F)

Model	Accuracy	Precision	Recall	F1	False alarms	Missed spam	5-fold CV F1
KNN (K=1)	1.000	1.000	1.000	1.000	0	0	1.0000
SVM (linear, C=0.01)	1.000	1.000	1.000	1.000	0	0	1.0000
Naive Bayes	0.999	0.995	1.000	0.997	1	0	0.9995

All three models rank every spam message above every legitimate message (ROC-AUC = 1.0), and tuned thresholds make them equivalent on this test set. At the default threshold KNN and the SVM tie for the highest precision (1.000) and recall (1.000); Naive Bayes ties on recall but has the lowest precision. Accuracy on clean data cannot separate the models, so a label-noise robustness test was added: a share of training labels was flipped and the models scored on clean test labels.

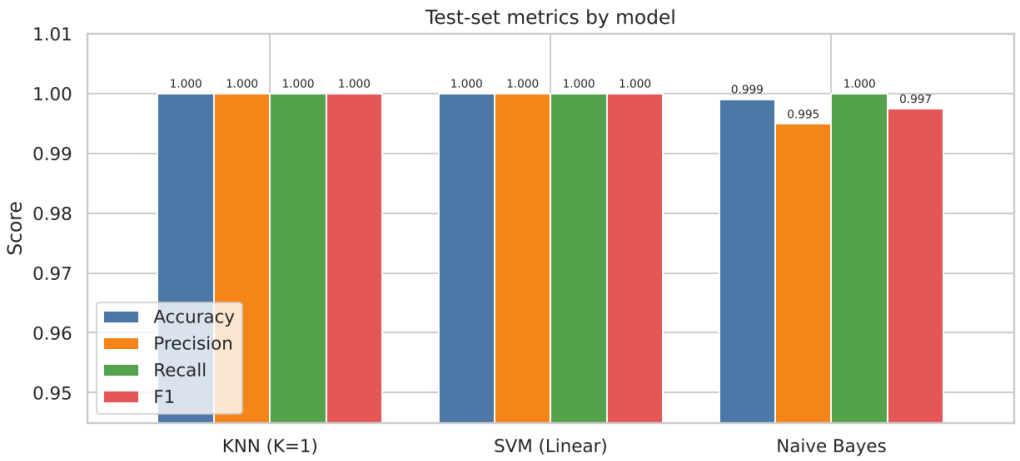


Figure 4. Test-set metrics for the three final models.

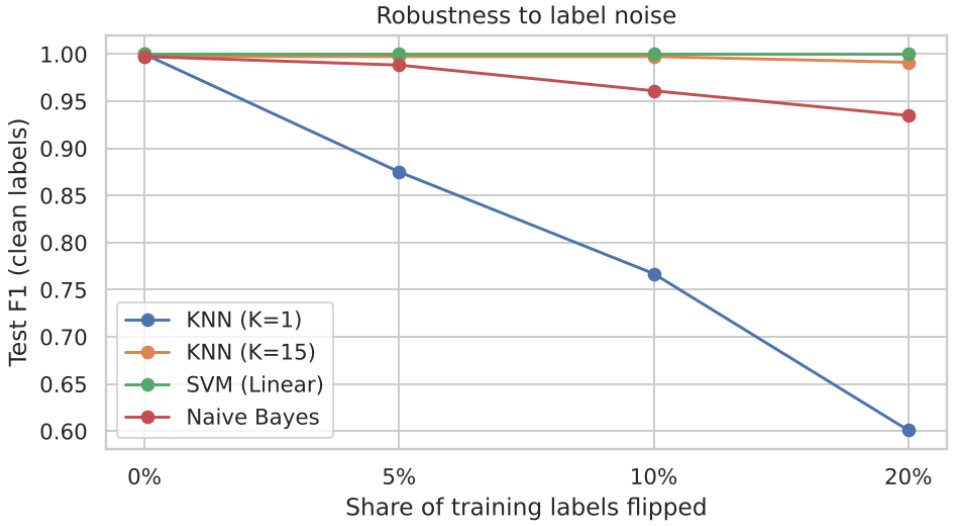


Figure 5. Test F1 when training labels are randomly flipped (mean of 5 seeds). K=1 KNN collapses; the linear SVM is unaffected; K=15 KNN and Naive Bayes degrade slowly.

Test F1 by label noise	0%	5%	10%	20%
KNN (K=1)	1.000	0.875	0.767	0.601
KNN (K=15)	0.997	0.997	0.997	0.991
SVM (linear)	1.000	1.000	1.000	1.000
Naive Bayes	0.997	0.988	0.961	0.935

Best model for high precision (almost never block a legitimate message): the linear SVM and KNN tie at 1.000; the SVM is preferable because it stays at 1.000 under label noise, while K=1 KNN and Naive Bayes lose precision (0.48 and 0.88 at 20% noise). **Best model for high recall** (almost never miss spam): all models reach 1.000 on clean data and recall stays at or above 0.99 for the SVM, K=15 KNN and Naive Bayes under noise; Naive Bayes keeps recall 1.000 at every noise level but pays for it with false alarms.

5. Final Analysis and Recommendation (Part G)

Model	Strengths	Weaknesses
KNN	No training; simple; explains decisions by similar example messages; non-parametric	Needs scaling; sensitive to K, metric and label noise (K=1); slower predictions; coarse scores
SVM	Maximum-margin solution; robust to noisy labels with small C; compact model (support vectors); kernels available	Needs scaling and tuning; no native probabilities; non-linear kernels are hard to interpret

Model	Strengths	Weaknesses
Naive Bayes	Fastest and smallest; probabilistic output; each feature's contribution can be read off; no scaling	Independence and Gaussian assumptions violated; over-confident probabilities; lowest precision here

Impact of the probability assumptions in Naive Bayes. Features are clearly correlated within each class (message length and URL count r = 0.77 in legitimate messages; length and word count r = 0.73 in spam), so evidence is counted more than once and posteriors are pushed toward 0 or 1: 99.9% of test messages receive a probability below 0.01 or above 0.99. Gaussian likelihoods fit count features such as special characters poorly; their near-zero variance in the legitimate class makes any deviation astronomically unlikely (log-likelihood about -14,027), so one feature can override all others. The prior (18.7% spam) pulls borderline cases toward legitimate. The ranking of messages is nevertheless perfect (ROC-AUC 1.0), so calibration or a better threshold would remove the single error.

Interpretability versus performance. Here there is no trade-off to pay: the transparent models (Naive Bayes, linear SVM) perform as well as or better than the opaque ones. Naive Bayes is the most transparent; the linear SVM combines readable weights with the best robustness; KNN explains by example but is the most sensitive to scaling, K and noise. RBF and polynomial kernels add complexity with no gain because the classes are linearly separable.

Business recommendation. Deploy the linear SVM (C about 0.01, standardised features) as the primary filter. Keep Gaussian Naive Bayes as a fast, explainable secondary signal, calibrated or with a raised threshold. Use KNN only with K around 15 and scaling, as an example-based explanation tool. Set the decision threshold by the business cost of a false alarm (blocking a real message) versus a miss (letting spam through); this data allows both to be zero, real data will not. Re-validate on messy, real, adversarial data before release and monitor feature drift, because all models rely mainly on *num_special_chars* (permutation importance: F1 drop 0.33 for the SVM), a feature a spammer could simply stop using.

Conclusion. All three families classify this dataset almost perfectly (F1 0.997 to 1.000) because it is cleanly separable. The implementations were verified (the hand-written Bayes calculation matches scikit-learn), and the models are distinguished by robustness and operating cost rather than accuracy: the regularised linear SVM is the strongest overall choice, Naive Bayes the most interpretable, and KNN the most fragile unless K and scaling are tuned.